7. Keras

Introduction

Keras is a high-level Deep Learning API integrated with TensorFlow. It provides a simple, user-friendly interface for building, training, and evaluating neural network models. Keras supports different types of neural networks, including Artificial Neural Networks (ANN), Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and Long Short-Term Memory (LSTM) networks. It offers pre-built layers, activation functions, optimizers, and loss functions that simplify model development. Keras is widely used in education, research, and industrial AI applications because of its simplicity, flexibility, and modular design. It enables developers to build, train, evaluate, and deploy powerful Deep Learning models with only a few lines of Python code.

Import Library

```
from tensorflow import keras
```

Common Functions

1. **Sequential()** – Creates a sequential neural network model.
2. **Dense()** – Adds a fully connected (dense) layer.
3. **Input()** – Defines the input layer.
4. **Dropout()** – Prevents overfitting by randomly dropping neurons.
5. **Flatten()** – Converts multidimensional input into a single dimension.
6. **Activation()** – Applies an activation function.
7. **compile()** – Configures the model for training.
8. **fit()** – Trains the model.
9. **evaluate()** – Evaluates the model performance.
10. **predict()** – Predicts output for new data.
11. **summary()** – Displays the model architecture.
12. **save()** – Saves the trained model.
13. **load_model()** – Loads a saved model.
14. **get_weights()** – Returns model weights.
15. **set_weights()** – Sets model weights.
16. **add()** – Adds a layer to the model.
17. **pop()** – Removes the last layer.
18. **BatchNormalization()** – Normalizes layer outputs.
19. **Conv2D()** – Adds a 2D convolution layer.
20. **MaxPooling2D()** – Adds a max pooling layer.
21. **SimpleRNN()** – Adds a Simple RNN layer.
22. **LSTM()** – Adds an LSTM layer.
23. **Embedding()** – Converts integer inputs into dense vectors.
24. **EarlyStopping()** – Stops training when performance stops improving.
25. **ModelCheckpoint()** – Saves the best model during training.

```
from tensorflow import keras
from tensorflow.keras.models import load_model
from tensorflow.keras.layers import (
    Dense, Input, Dropout, Flatten,
    Activation, BatchNormalization,
    Conv2D, MaxPooling2D,
    SimpleRNN, LSTM, Embedding
)
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
import numpy as np

# 1. Sequential()
model = keras.Sequential()
print("1. Sequential Model Created")

# 2. Dense()
model.add(Dense(8, input_shape=(4,)))
print("2. Dense Layer Added")

# 3. Input()
model2 = keras.Sequential([
    Input(shape=(4,)),
```

```
    Dense(8)
])
print("3. Input Layer Created")

# 4. Dropout()
model.add(Dropout(0.2))
print("4. Dropout Layer Added")

# 5. Flatten()
model3 = keras.Sequential([
    Flatten(input_shape=(2,2))
])
print("5. Flatten Layer Added")

# 6. Activation()
model.add(Activation("relu"))
print("6. Activation Layer Added")

# 7. compile()
model.add(Dense(1))
model.compile(optimizer="adam",
              loss="mse",
              metrics=["mae"])
print("7. Model Compiled")

# Sample Data
X = np.random.rand(20,4)
Y = np.random.rand(20)

# 8. fit()
model.fit(X,Y,epochs=1,verbose=0)
print("8. Model Trained")

# 9. evaluate()
print("9. Evaluation:", model.evaluate(X,Y,verbose=0))

# 10. predict()
print("10. Prediction:\n", model.predict(X[:2],verbose=0))

# 11. summary()
print("11. Model Summary")
model.summary()

# 12. save()
model.save("keras_model.keras")
print("12. Model Saved")

# 13. load_model()
loaded_model = load_model("keras_model.keras")
print("13. Model Loaded")

# 14. get_weights()
weights = model.get_weights()
print("14. Number of Weight Arrays:", len(weights))

# 15. set_weights()
loaded_model.set_weights(weights)
print("15. Weights Set")

# 16. add()
model.add(Dense(2))
print("16. New Layer Added")

# 17. pop()
model.pop()
print("17. Last Layer Removed")

# 18. BatchNormalization()
bn_model = keras.Sequential([
    Dense(8,input_shape=(4,)),
    BatchNormalization()
])
print("18. BatchNormalization Added")

# 19. Conv2D()
cnn = keras.Sequential([
    Conv2D(8,(3,3),input_shape=(28,28,1))
])
print("19. Conv2D Layer Added")

# 20. MaxPooling2D()
cnn.add(MaxPooling2D((2,2)))
print("20. MaxPooling2D Added")
```

```
# 21. SimpleRNN()
rnn = keras.Sequential([
    SimpleRNN(8,input_shape=(5,3))
])
print("21. SimpleRNN Added")

# 22. LSTM()
lstm = keras.Sequential([
    LSTM(8,input_shape=(5,3))
])
print("22. LSTM Added")

# 23. Embedding()
embed = keras.Sequential([
    Embedding(input_dim=100,
              output_dim=8,
              input_length=10)
])
print("23. Embedding Layer Added")

# 24. EarlyStopping()
early = EarlyStopping(
    monitor="loss",
    patience=2
)
print("24. EarlyStopping Callback Created")

# 25. ModelCheckpoint()
checkpoint = ModelCheckpoint(
    "best_model.keras",
    save_best_only=True
)
print("25. ModelCheckpoint Created")
```

```
1. Sequential Model Created
2. Dense Layer Added
3. Input Layer Created
4. Dropout Layer Added
5. Flatten Layer Added
6. Activation Layer Added
7. Model Compiled
8. Model Trained
9. Evaluation: [0.6089860200881958, 0.7325158715248108]
10. Prediction:
 [[-0.04936888]
 [ 0.00571147]]
11. Model Summary
```

Model: "sequential_8"

Layer (type)	Output Shape	Param #
dense_5 (Dense)	(None, 8)	40
dropout_1 (Dropout)	(None, 8)	0
activation_1 (Activation)	(None, 8)	0
dense_7 (Dense)	(None, 1)	9

Total params: 149 (600.00 B)
Trainable params: 49 (196.00 B)
Non-trainable params: 0 (0.00 B)
Optimizer params: 100 (404.00 B)

```
12. Model Saved
13. Model Loaded
14. Number of Weight Arrays: 4
15. Weights Set
16. New Layer Added
17. Last Layer Removed
18. BatchNormalization Added
19. Conv2D Layer Added
20. MaxPooling2D Added
21. SimpleRNN Added
22. LSTM Added
23. Embedding Layer Added
24. EarlyStopping Callback Created
```

Start coding or generate with AI.